ArgoCon EU 2026

Don’t Do What Donny Don’t Does:

How to set up your cluster for Workflows success

Becky Pauley
tailscale

Tim Collins
pipekit

Smarch 23, 2026

Intros

Becky Pauley

Member of Technical Staff @ Tailscale

Building the Tailscale Kubernetes Operator 🏗️

Argo enthusiast

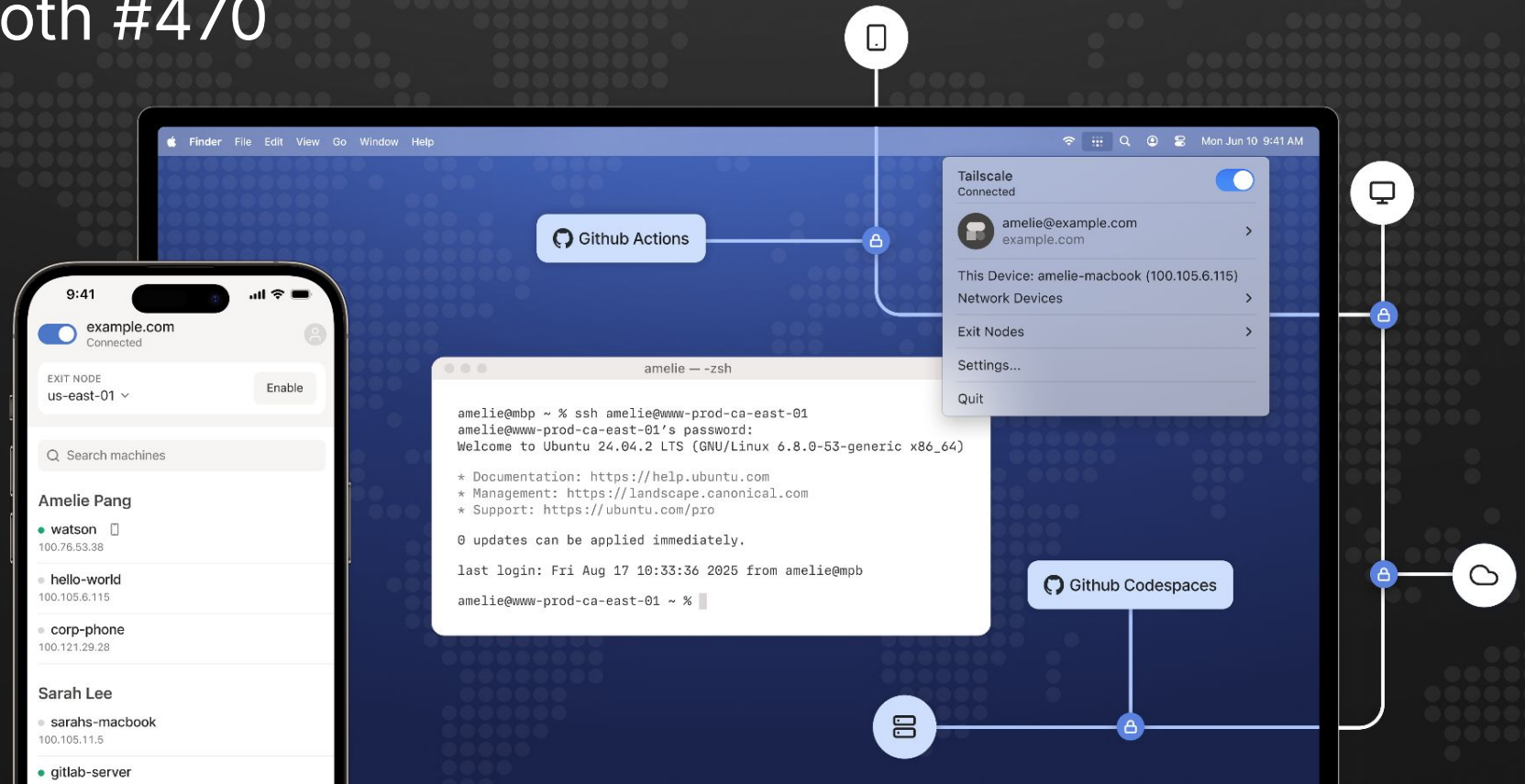
My hobby is falling off things 🏄‍♀️🛹🧗‍♀️

Knows nothing about the Simpsons 😱

tailscale

The secure connectivity platform for modern development teams

Booth #470



Intros

Tim Collins

Staff Infrastructure Engineer @ Pipekit.io

Argo Maintainer.

Co-author of the argocd-lovely-plugin.

The one on the left in the picture.

pipekit

Scale Argo Workflows with Pipekit

- **Direct support** from **Argo Workflows maintainers**
- Add **3 Argo maintainers** and **8 Argo contributors** to your team
- Save time debugging Argo Workflows with **Pipekit's Multi-cluster Control Plane UI**
- Serving **startups & Fortune 500** enterprises since 2021:

Enterprise Support for Argo
Ideal for Platform Eng & Data/ML teams scaling with Argo

Control Plane for Argo Workflows
Unlock granular RBAC, multi-cluster architectures, and observability

Image credit: Phil Botha, Unsplash

Do: Remember the Architecture

Workflow Controller

Manages workflows.

Argo Server

UI and API for Argo Workflows.

Talk: Demystifying Argo Workflows
github.com/pipekit/talk-demos/tree/main/argocon-demos/2024-demystifying-argo-workflows-an-architectural-deep-dive

Don’t forget:
It’s all just Kubernetes

Argo Workflows is a Kubernetes Operator

Orchestrates pods

You’re going to need to know Kubernetes

Do: Observe your cluster

Health

- Workflow Controller
- Cluster
- Workflows steps (pods)
- Any dependent tooling (database, object storage, disks etc)

Log Aggregation

- Use the links feature in the UI to link out to your observability tools

argo-workflows.readthedocs.io/en/latest/configure-archive-logs

Don’t: Ignore the resources your pods need!

Do: Set Requests and Limits

Requests

Baseline or average resources required to run. Used for scheduling.

Limits

The maximum level of memory/CPU the container/pod can consume.

Note:
Requests and Limits can now also be set at the Pod level in Kubernetes - Argo Workflows supports Container level requests and limits.

Don't: Forget Controller and Server resources

Workflow Controller

Manages workflows.

Argo Server

UI and API for Argo Workflows.

Do: Check defaults in workflow-controller-configmap

```
# Defaults for main containers. These can be overridden by the template.
 mainContainer: |
  imagePullPolicy: IfNotPresent
  resources:
    requests:
      cpu: 0.1
      memory: 64Mi
    limits:
      cpu: 0.5
      memory: 512Mi
```

Set sensible defaults in workflow-controller-configmap

Override in individual templates when defaults don't suit

Do:
Keep things tidy

- TTL strategy
- Workflow Archiving
- Garbage Collection

Do:
Check your versions

Don’t: be insecure

Do: Use SSO

Benefit from SSO RBAC

See
https://argo-workflows.readthedocs.io/en/latest/argo-server-sso

Don't: leave Argo Server open to the public internet

```
kubectl -n argo port-forward svc/argo-workflows-server 2746:2746
```

🙁

Image credit: Hannah Wernecke, Unsplash

Example: Connecting to Argo Server securely

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name:  argo-workflows-server
 namespace: argo
spec:
 ingressClassName: tailscale
 tls:
   - hosts:
       - argo-workflows-server # The desired MagicDNS
name
 rules:
 - http:
     paths:
     - path: /
       pathType: Prefix
       backend:
         service:
           name: argo-workflows-server
           port:
             number: 2746
```

github.com/pipekit/talk-demos/

Do: Get permissions right

Do: set your cluster up for success

Autoscaling

ARM64 vs AMD64

SPOT vs ONDEMAND

Do: Prepare for massive scale

Consider begging for a bigger control plane.

Consider splitting into multiple Kubernetes clusters.

Do: Prepare for massive scale

Workflow Controller

Indicative numbers for approximately **10,000 concurrent Workflows**:

256	100	200
workflow-workers	QPS	burst
16	64Gi	
CPU	RAM	

Do set

- A sensible default requeue time.
- One controller replica (leader election).

Do: Prepare for massive scale

Argo Server

Multiple Replicas

Consider UI, API, CLI responsiveness.

Archiving workflows

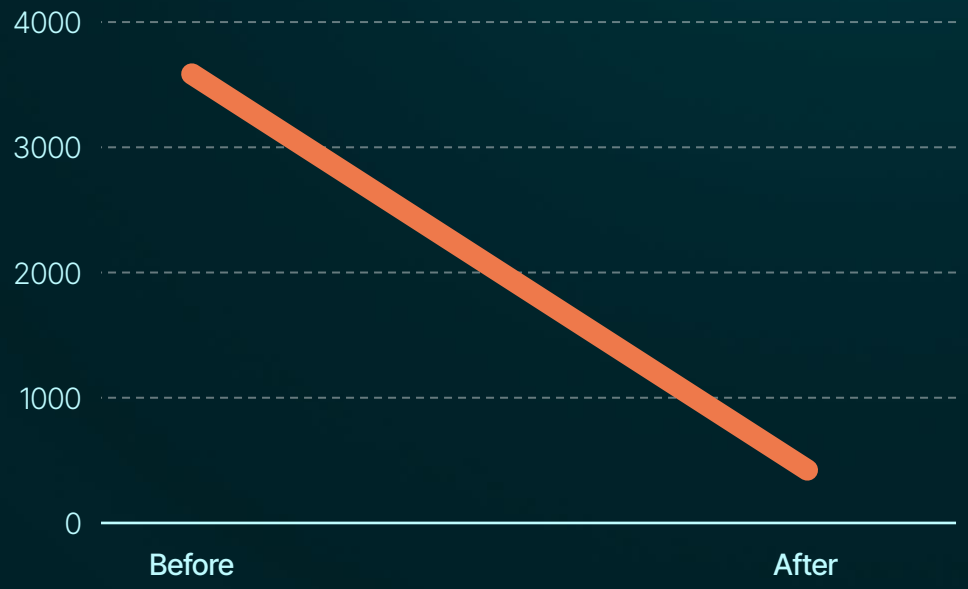
The UI sucks at scale

Don't: feel forced to use YAML

Golang	github.com/argoproj/argo-workflows/tree/v4.0.1/sdks/go
Java	github.com/argoproj/argo-workflows/tree/v4.0.1/sdks/java
Python (Hera)	hera-proj.io
Typescript (Juno)	github.com/lumindigital/juno

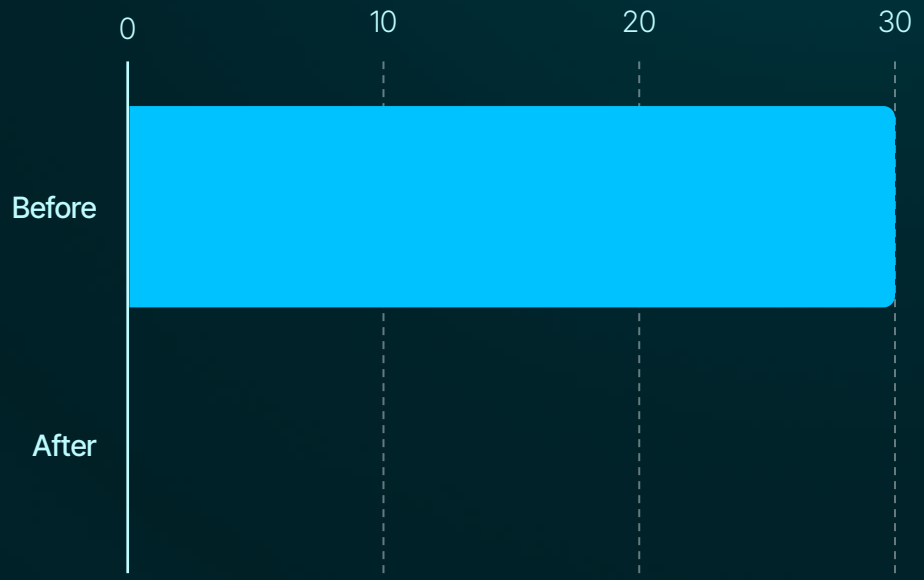
Executives. Don't ignore this advice

Average Workflow runtime **from 6 hours to 30 minutes.**



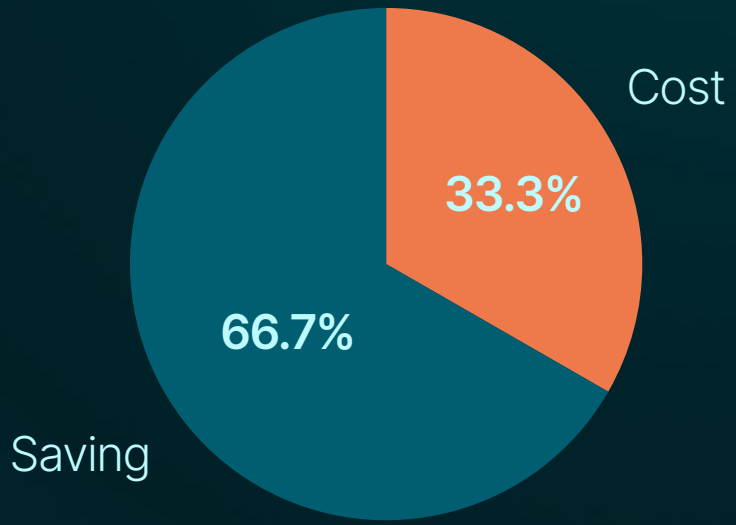
Executives. Don't ignore this advice

30% Workflow failure → **99.9% reliability**



Executives. Don’t ignore this advice

2/3 reduction in cloud compute costs.



Conclusion

Do remember the architecture
Don't forget that it's all just Kubernetes
Do observe your cluster
Do consider log management
Don't ignore pod resources
Do set requests and limits
Don't forget controller and server resources
Do check defaults in the workflow-controller-configmap
Do keep things tidy
Do check your versions
Don't be insecure
Do use SSO
Don't leave Argo open to the internet
Do get permissions right
Do set your cluster up for success
Do prepare for massive scale
Do set only one controller replica
Do set a sensible default requeue time
Don't feel forced to use YAML
Don't ignore this advice

Thank You!

This talk & examples:

github.com/pipekit/talk-demos

Come and talk to us:

Booth 470

tailscale

Booth 494

pipekit

The Simpsons images and their derivatives used in this presentation are copyright 20th Television / Disney